

When SEARCH may not solve

Limits of Blind Search

Simple search algorithms often struggle with complex constraints where the state space explodes. In the 8×8 grid domino problem, blind search is computationally inefficient.

Power of Representation

The "right representation" can solve problems instantly. By coloring the grid, we see each domino covers one black and one red square, making the "mutilated grid" proof trivial.

When SEARCH may not solve

The Goal

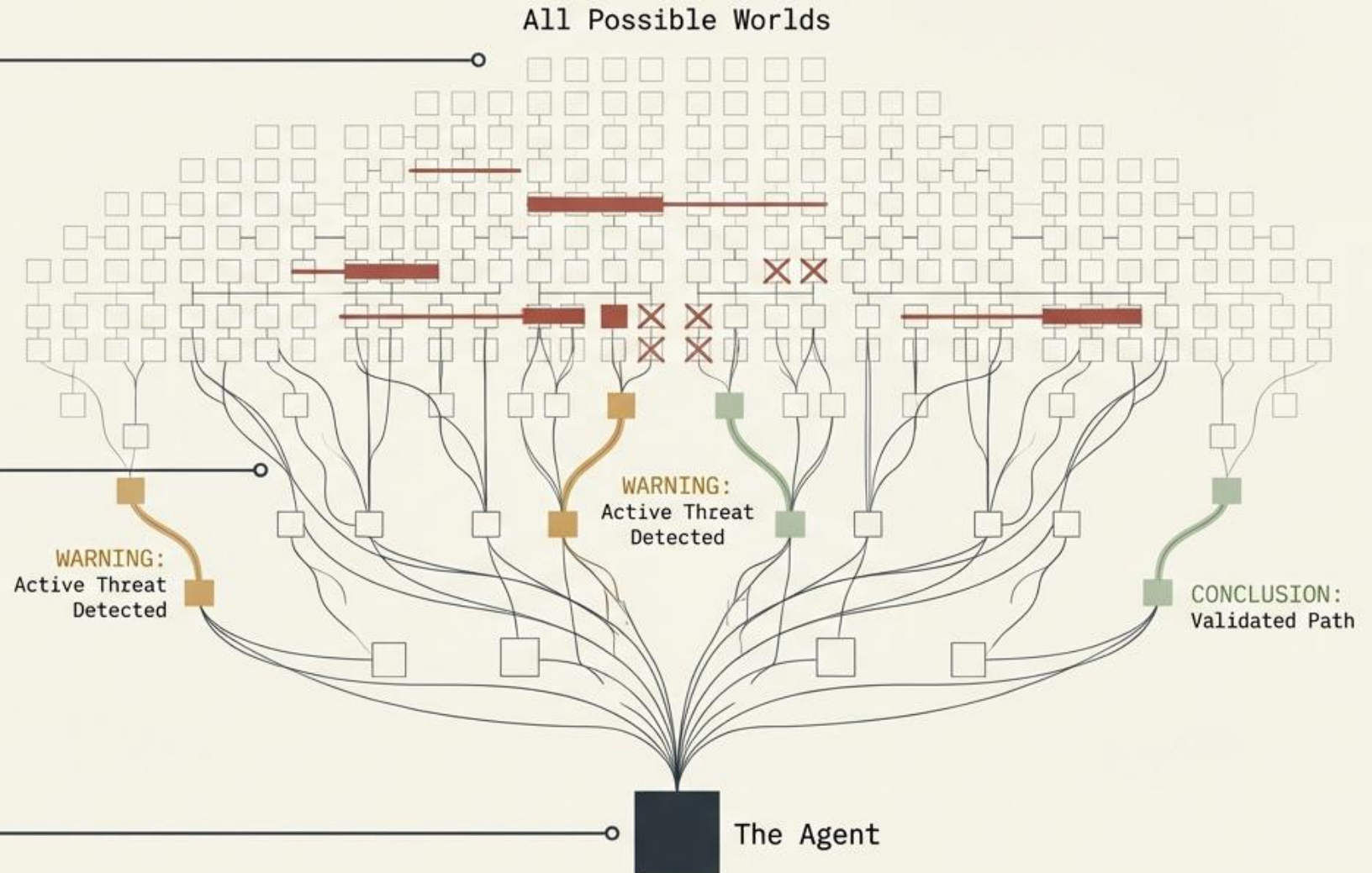
The AI acts as a detective. It does not construct the truth; it methodically subtracts the impossible until only the truth remains.

The Variables

Every binary feature of the world (a pit, a wall) doubles the number of possible realities.

The Unknown

An AI agent wakes up blind in a new environment. Before a single observation is made, every conceivable reality is possible.



Limitation of Problem-solving Agents

Hidden Knowledge

Their understanding of the world is often buried in domain-specific code (e.g., a RESULT function), meaning they can't deduce general truths about their environment.

Atomic Constraints

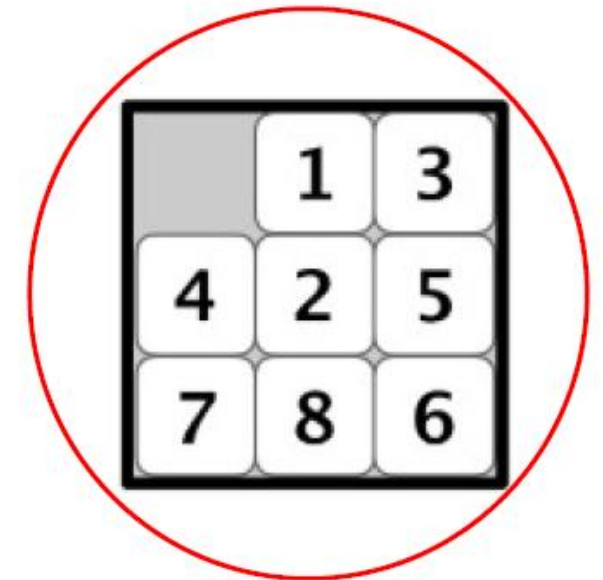
They struggle in partially observable or large environments because they represent states as "atomic" entities, making it impossible to track all possibilities efficiently.

Limitation of Problem-solving Agents

Hidden Knowledge

Their understanding of the world is often buried in domain-specific code (e.g., a RESULT function), meaning they can't deduce general truths about their environment.

An 8-puzzle agent knows that "Action: Move Tile Up" results in a new state. However, it **cannot "know"** or deduce a general rule like *"Two tiles cannot occupy the same square."* To the agent, every state is just a unique ID; it can't reason about the physics or constraints of the board itself.



Initial State
(aka. Start State)



Limitation of Problem-solving Agents

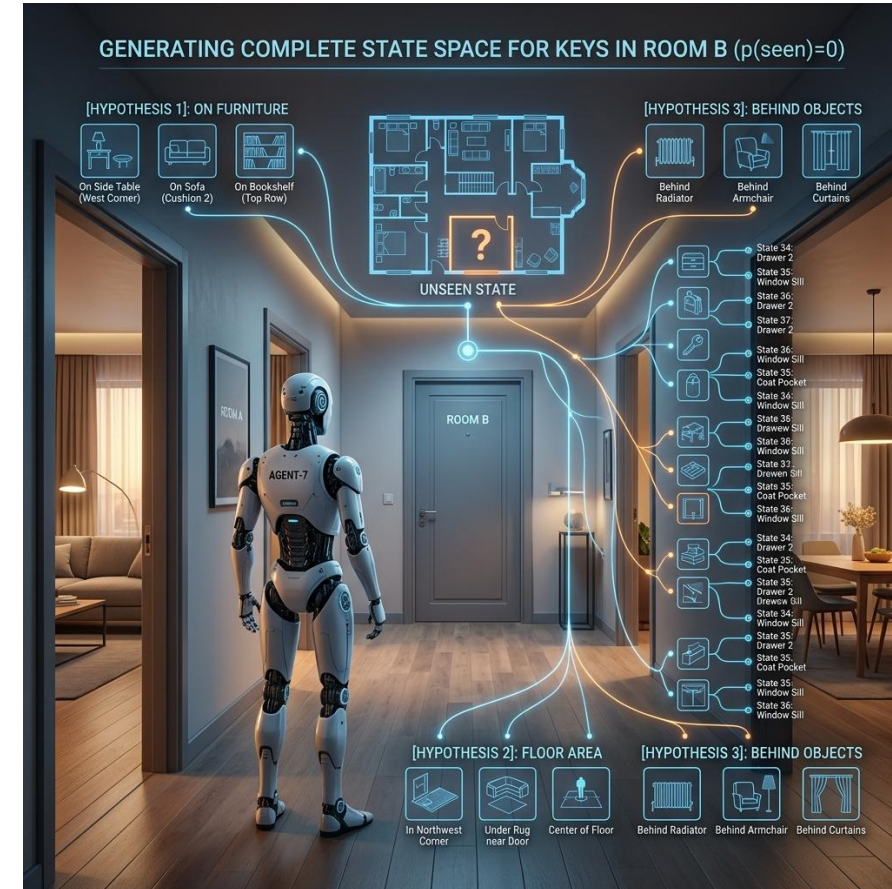
Atomic Constraints

They struggle in partially observable or large environments because they represent states as "atomic" entities, making it impossible to track all possibilities efficiently.

Imagine a robot looking for keys in a house with 10 rooms. If it can't see into Room B, a problem-solving agent might have to generate every possible concrete state where the keys could be (Keys in Room B, corner 1; Keys in Room B, corner 2, etc.).

The Consequence

In large environments, the number of possible states explodes (state-space explosion), making it "hopeless" to list them all.

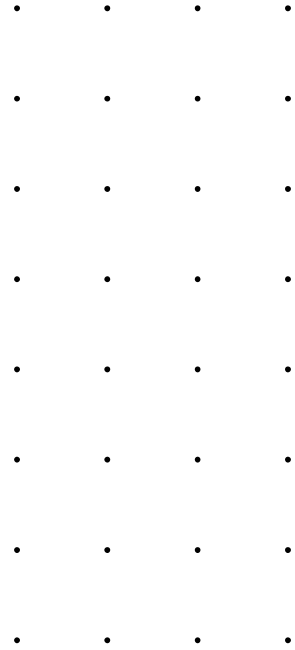


Week 8: Knowledge-based Agents (Logical Agents)

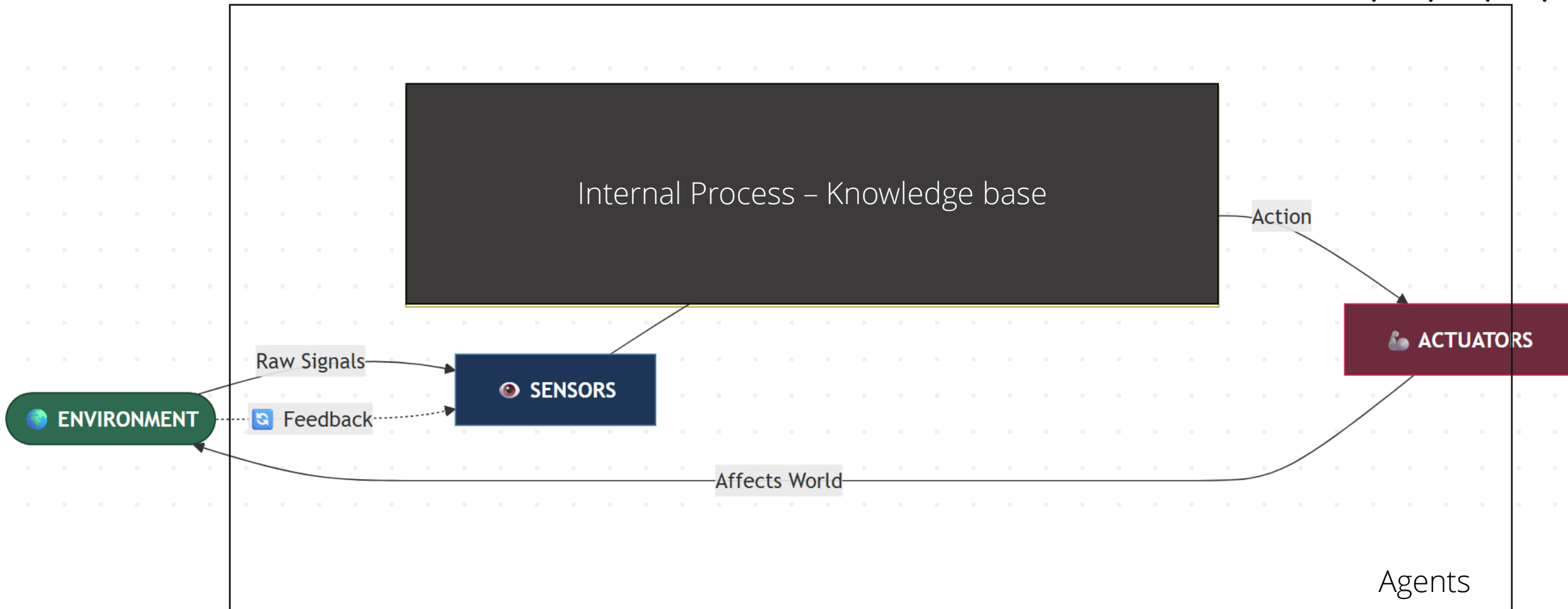


Overview

- Overall Agent design
- Example: Wumpus World
- General principles of logic



Design of Knowledge-based agent



Knowledge Base

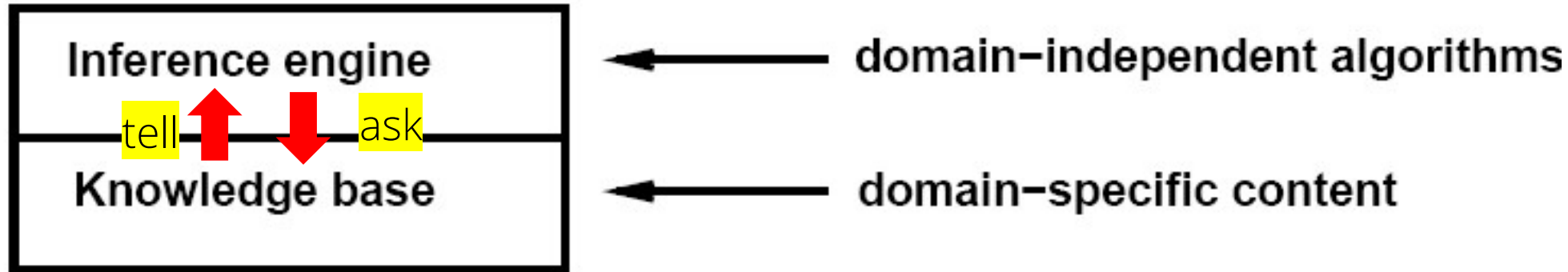
Knowledge Base: set of sentences represented in a knowledge, representation language and represents assertions about the world.

- The KB stores everything the agent knows about its environment, its rules, and its goals.



[illegible]

Knowledge-based agents



TELL

This is how the agent "learns" or "remembers." When it sees something **new** (a percept), it **TELLs the KB**. It also **TELLs the KB** which action it eventually took so it can keep track of its own history.

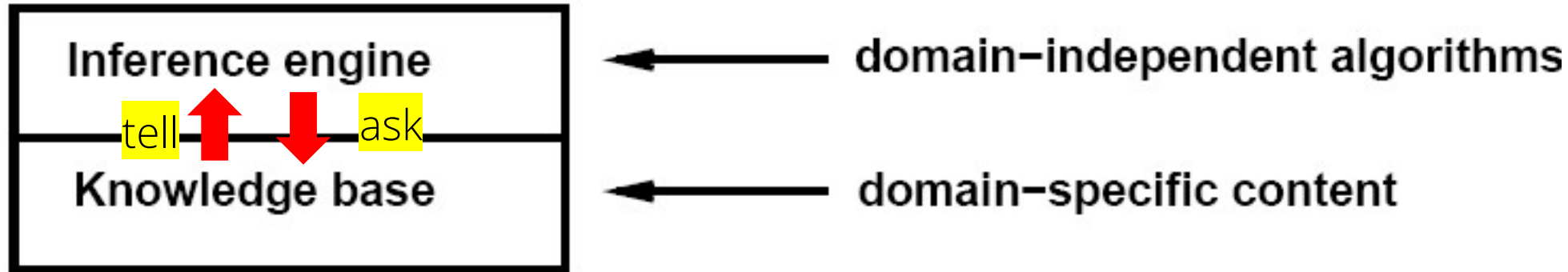
ASK

It asks the KB what to do. The KB doesn't just look up an answer; it uses Inference (reasoning) to figure out hidden facts (like "Smoke usually means fire, and fire is dangerous").

Inference rule

when one ASKs questions of the KB, the answer should follow from what has been TELLed to the KB previously

Knowledge-based agents



function KB-AGENT(*percept*) **returns** an *action*

persistent: *KB*, a knowledge base

t, a counter, initially 0, indicating time

TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))

action $\leftarrow$ ASK(*KB*, MAKE-ACTION-QUERY(*t*))

TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))

t $\leftarrow$ *t* + 1

return *action*

Given a percept, the agent

- Add the percept to its knowledge base,
- **Ask** the knowledge base for the best action,
- **Tell** the knowledge base that it has in fact taken that action.

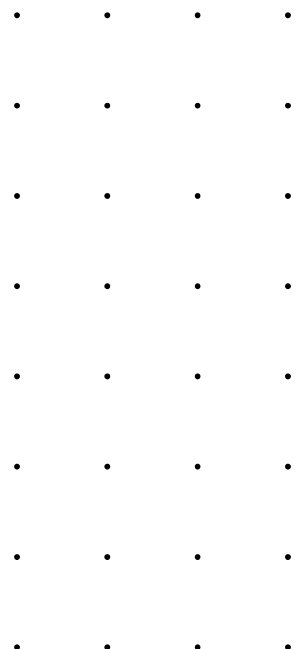
Knowledge-based agents

DECLARATIVE APPROACH

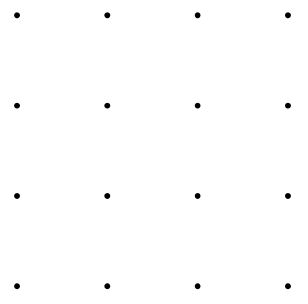
- A knowledge-based agent can be built simply by TELLing it what it needs to know.
- Starting with an empty knowledge base, the agent designer can TELL sentences one by one

PROCEDURAL APPROACH

Encodes desired behaviors directly as program code



Knowledge-based agents



CORE AGENT ABILITIES



Represent

Maintain an internal model of states and actions in a formal language.



Incorporate

Update the model with new percepts from the environment (TELL).



Deduce

Infer hidden properties and appropriate next steps (ASK).

.
.

WUMPUS WORLD ANALYSIS

Wumpus World



Environment Walkthrough

Performance measure

gold +1000, death -1000

-1 per step, -10 for using the arrow

Environment

Squares adjacent to wumpus are smelly

Squares adjacent to pit are breezy

Glitter iff gold is in the same square

Shooting kills wumpus if you are facing it

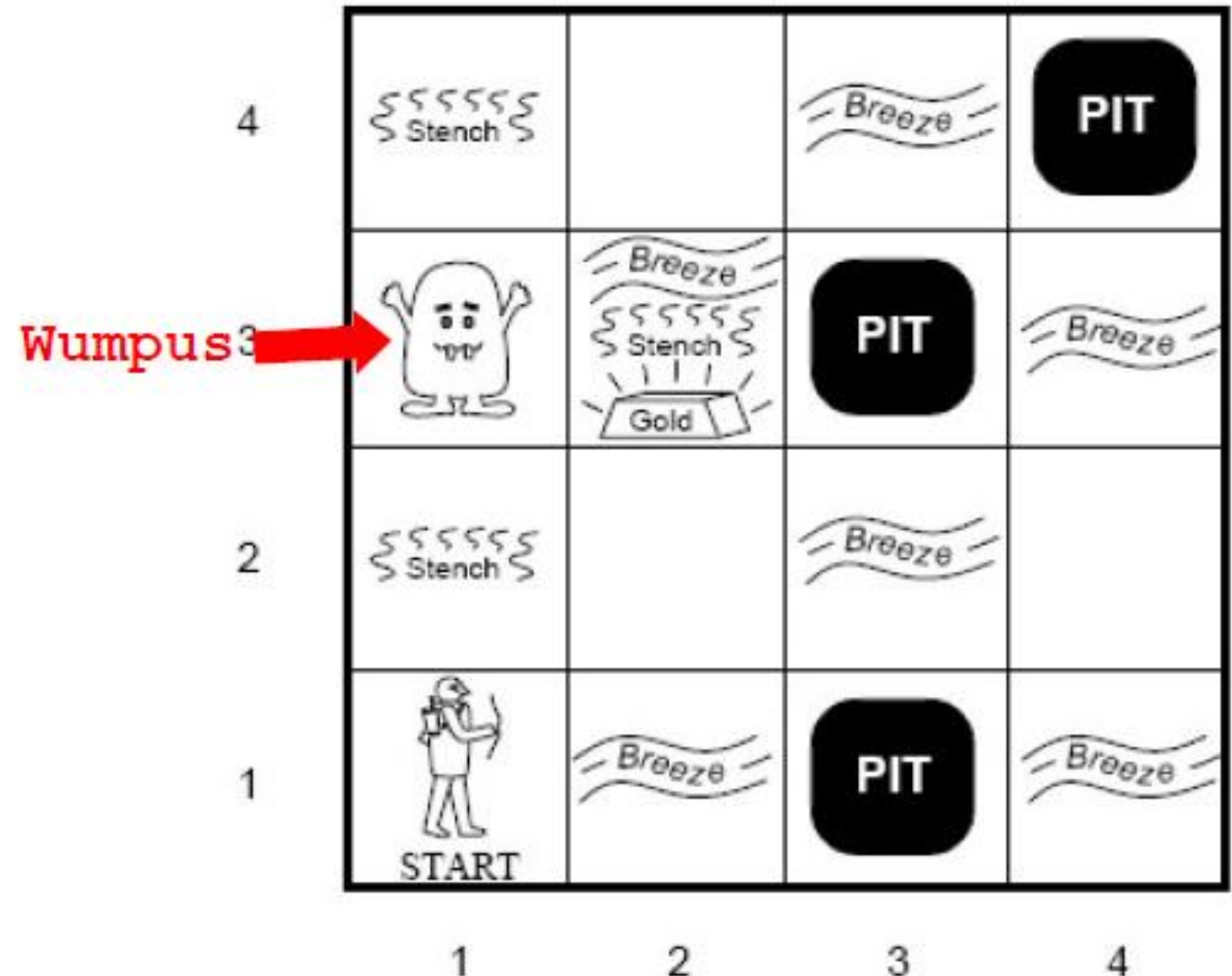
Shooting uses up the only arrow

Grabbing picks up gold if in same square

Releasing drops the gold in same square

Sensors Breeze, Glitter, Smell

Actuators Left turn, Right turn,
Forward, Grab, Release, Shoot



Environment Walkthrough

WUMPUS WORLD TRAITS



OBSERVABLE

NO

Only local perception is possible; the agent cannot see the entire grid at once.



DETERMINISTIC

YES

The outcomes of actions are exactly specified and predictable.



EPISODIC

NO

Sequential at the level of actions; current choices affect future states.



STATIC

YES

The Wumpus and pits do not move during the course of the agent's actions.



DISCRETE

YES

States, percepts, and actions are distinct and clearly defined.

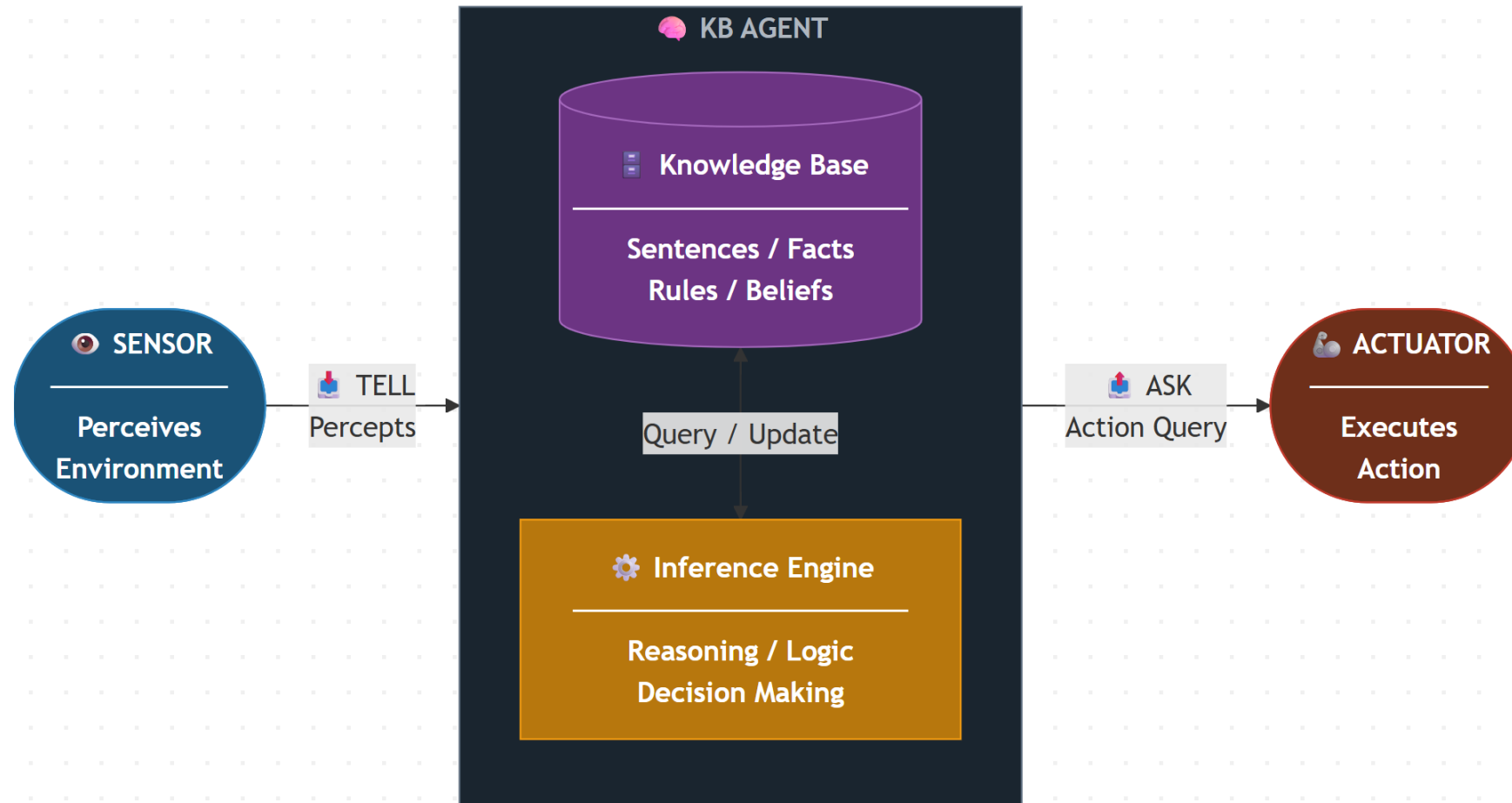


SINGLE-AGENT

YES

The Wumpus acts as a natural feature rather than a competing agent.

Environment Walkthrough



Environment Walkthrough

Initial Step [1,1]

Percept: [None, None, None, None, None].

Deduction: No Pit or Wumpus in [1,2] or [2,1]. These squares are **OK**.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

Environment Walkthrough

Next Step [2,1]

Percept: [Breeze, None, ...].
Deduction: Pit must be in [2,2] or [3,1]. Return to [1,1] and try [1,2].

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

Environment Walkthrough

Next Step [2,2]

By intersection of constraints, [2,2] is deduced as safe because there was no breeze in [1,2].

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

Environment Walkthrough

At square [2,3]

Percept[glitter , stench, breeze]

THUS pick up gold

Deduction: pit in [3,3] or [2,4]

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

Logic

- KB consists of sentences
- Sentences are physical configurations of the agent
 - Syntax: These are the "grammar rules." It specifies which combinations of symbols are "well-formed."

$$x + y = 4$$

$$x4y+ =$$

- Semantics: This is the "meaning." It defines the truth of a sentence in a specific situation

$$x + y = 4$$

$$x = 2 \text{ and } y = 2$$

$$x = 1 \text{ and } y = 1.$$

Logic – Model and possible worlds

- Logic uses the term Model to describe a "possible world" or a mathematical abstraction that fixes every fact as either true or false.
 - If a sentence α is true in a specific model $\mathbf{m}$, we say that $\mathbf{m}$ satisfies α .
 - The set of all possible models where a sentence is true is written as $M(\alpha)$.

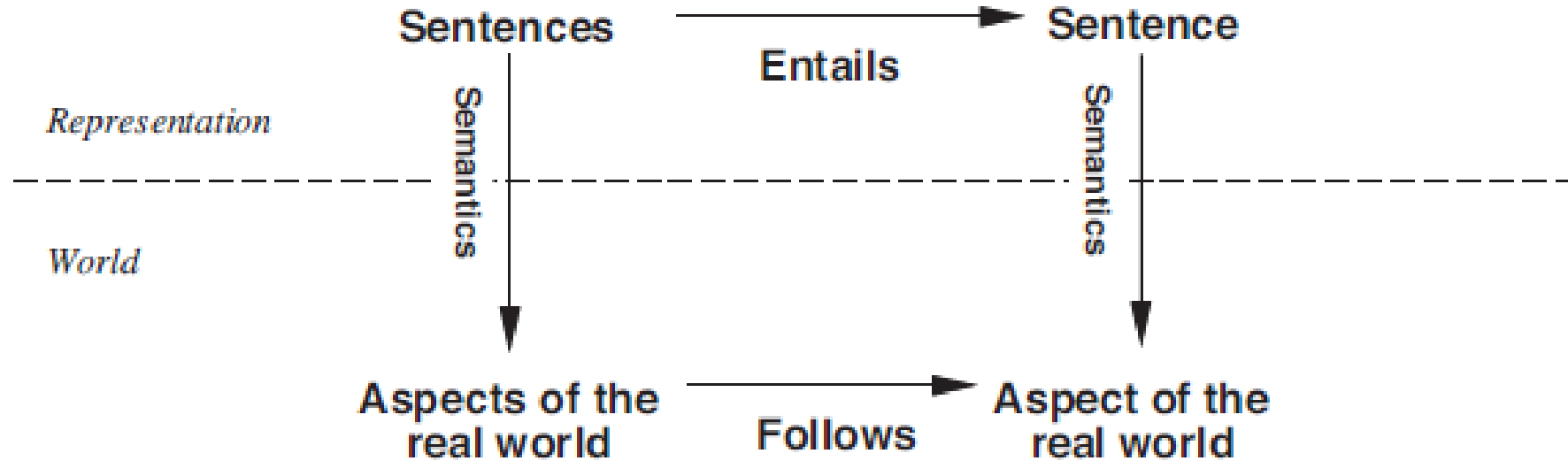
Logic – Logical Entailment $\alpha \models \beta$

- Entailment means that one sentence follows logically from another.
- Definition: α entails β if and only if in every possible world where α is true, β is also true.
- Example
 - If you know "It is raining α ", it entails "The ground is wet" β .
 - You don't need to see the ground; the logic guarantees it.

Logic – Inference

- **Entailment** is a relationship between sentences
- **Inference** is the actual process (or algorithm) used to find those relationships.
 - Soundness (Truth-Preserving): An inference algorithm is sound if it only derives sentences that are actually entailed. It doesn't "make things up."
 - Completeness: An algorithm is complete if it can find every sentence that is entailed by the knowledge base.
 - Model Checking: One way to do inference is to check every possible world to see if a conclusion holds true—this is called "model checking."

Logic



- Sentences are physical configurations of the agent
- Reasoning is a process of constructing new physical configurations from old ones.
- Logical reasoning should ensure that the new configurations represent aspects of the world that actually follow from the aspects that the old configurations represent.

Environment Walkthrough

8

Model Enumeration

For 3 boolean choices regarding pits in unknown squares, there are $2^3 = 8$ possible models.

Logic checks which models are consistent with current observations.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 A → A	2,1	3,1	4,1

- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus

Environment Walkthrough

- $P_{12}=\{T/F\}$ (there is/isn't a pit in cell [1,2])
- $P_{22}=\{T/F\}$ (there is/isn't a pit in cell [2,2])
- $P_{31}=\{T/F\}$ (there is/isn't a pit in cell [3,1])

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 A → A	2,1	3,1	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

Environment Walkthrough

. . . .
. . . .
. . . .

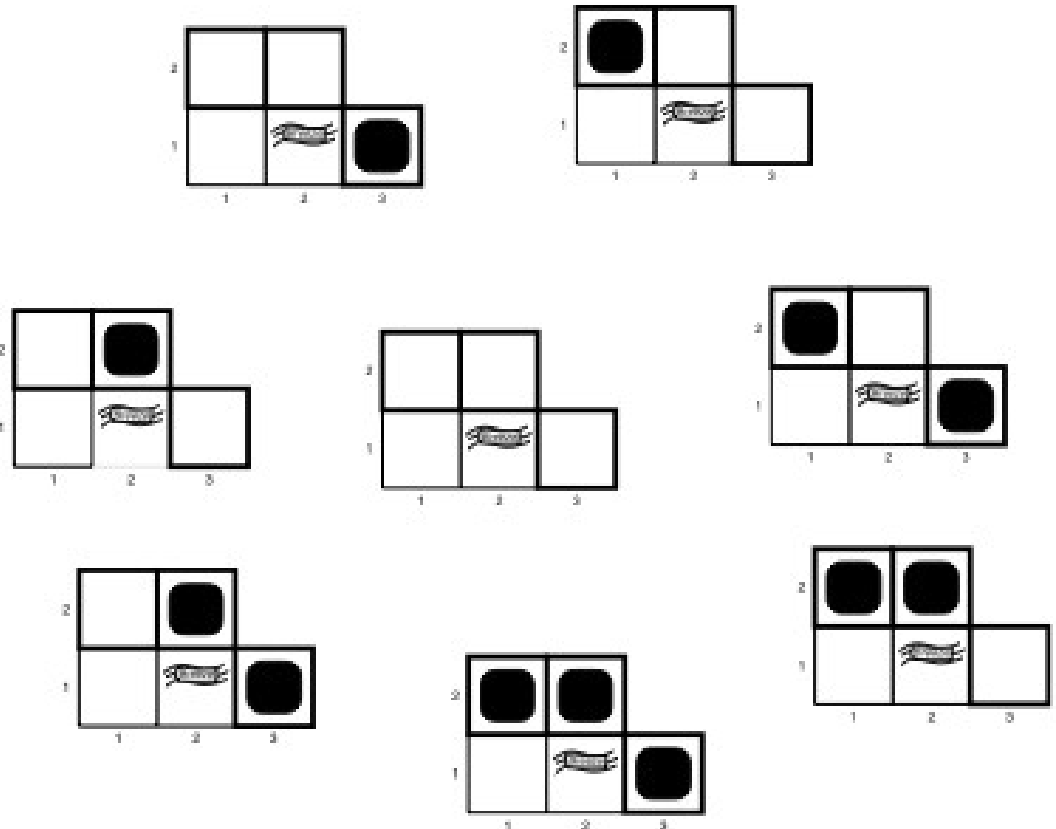
	P12	P22	P31
Model 1	F	F	F
Model 2	F	F	T
Model 3	F	T	F
Model 4	F	T	T
Model 5	T	F	F
Model 6	T	F	T
Model 7	T	T	F
Model 8	T	T	T

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 A → A	2,1	3,1	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

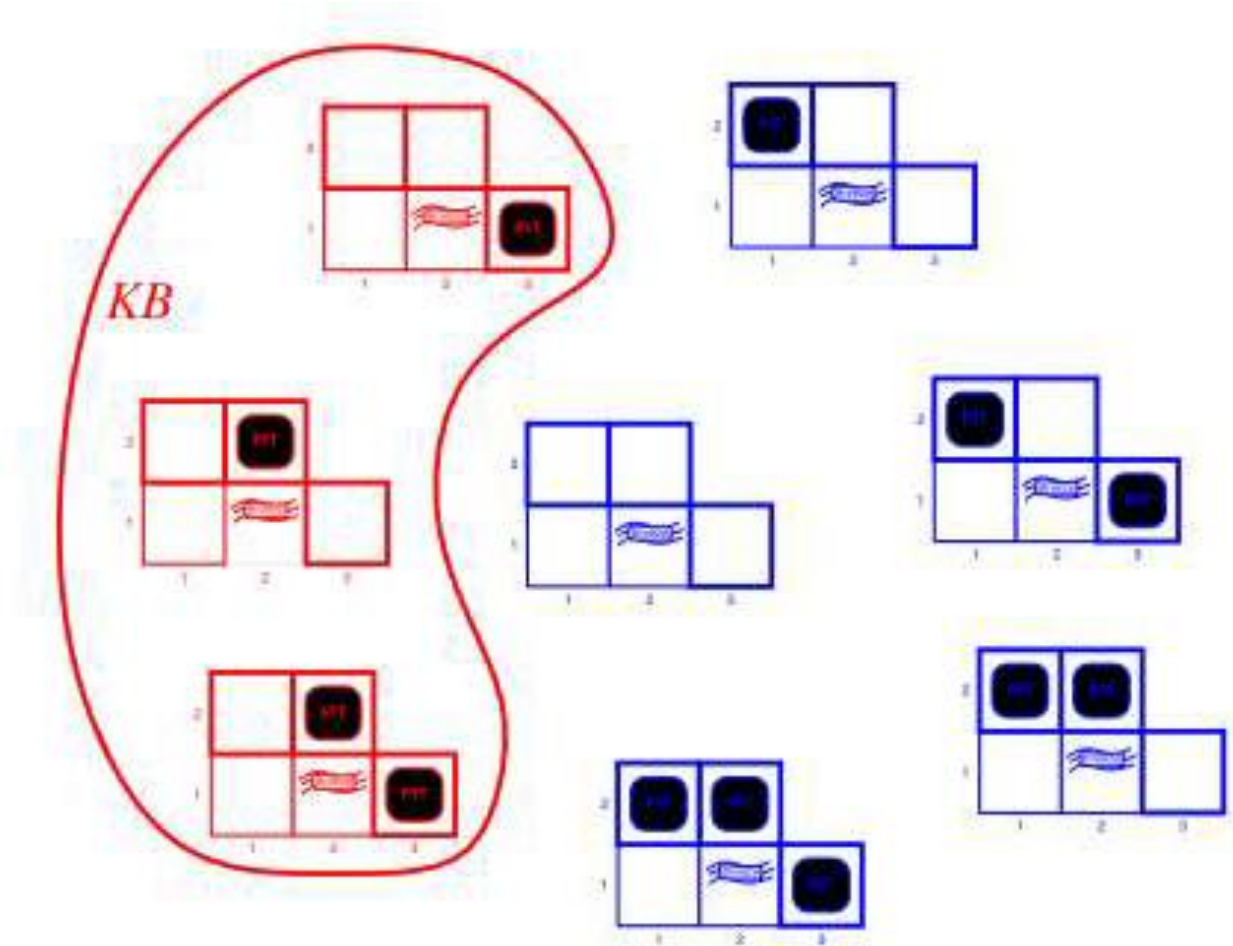
Environment Walkthrough

	P12	P22	P31
Model 1	F	F	F
Model 2	F	F	T
Model 3	F	T	F
Model 4	F	T	T
Model 5	T	F	F
Model 6	T	F	T
Model 7	T	T	F
Model 8	T	T	T



WUMPUS World Model

KB = world rules + observation

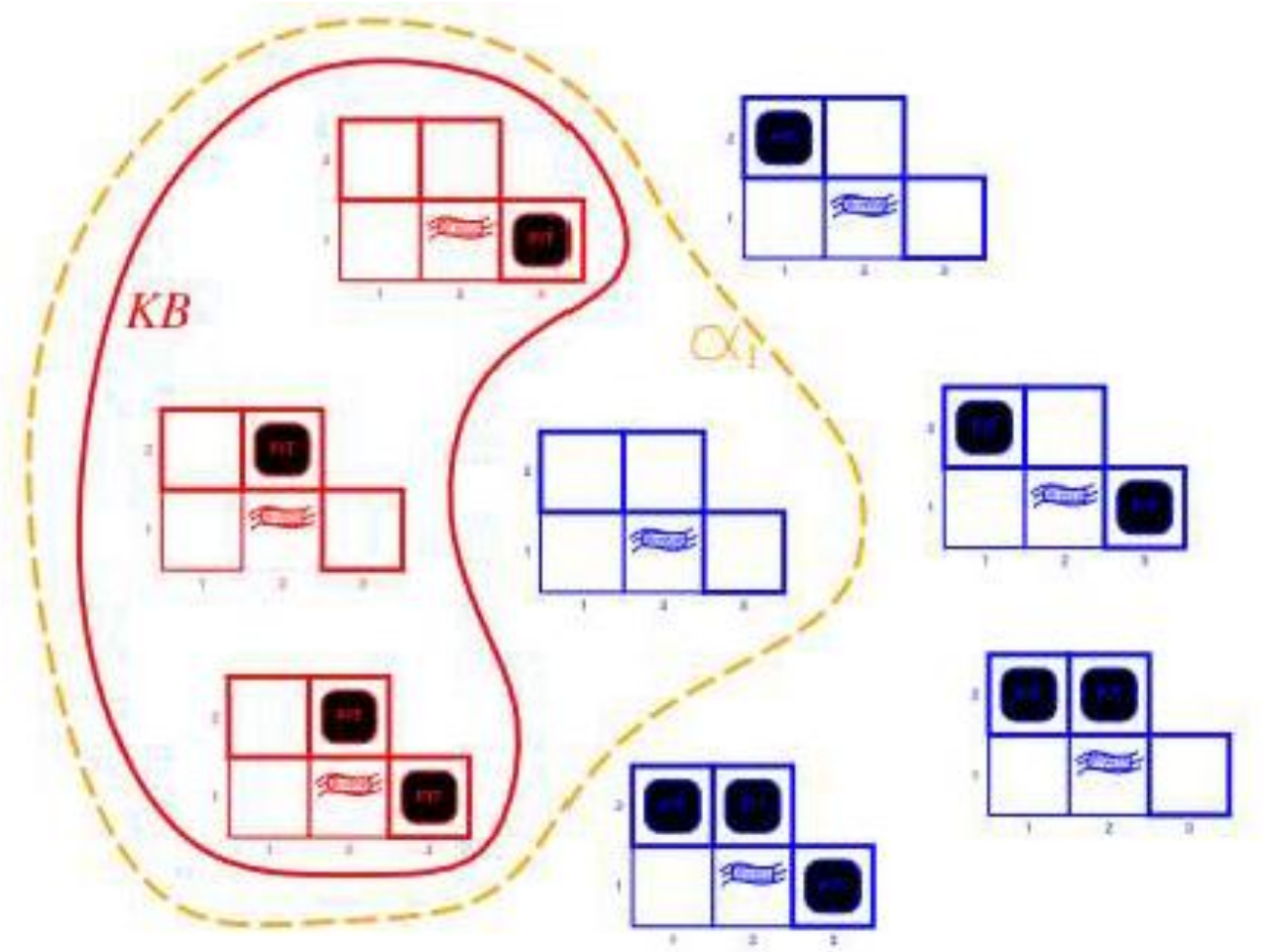


WUMPUS World Model

KB = world rules + observation

α_1 = "[1,2] is safe"

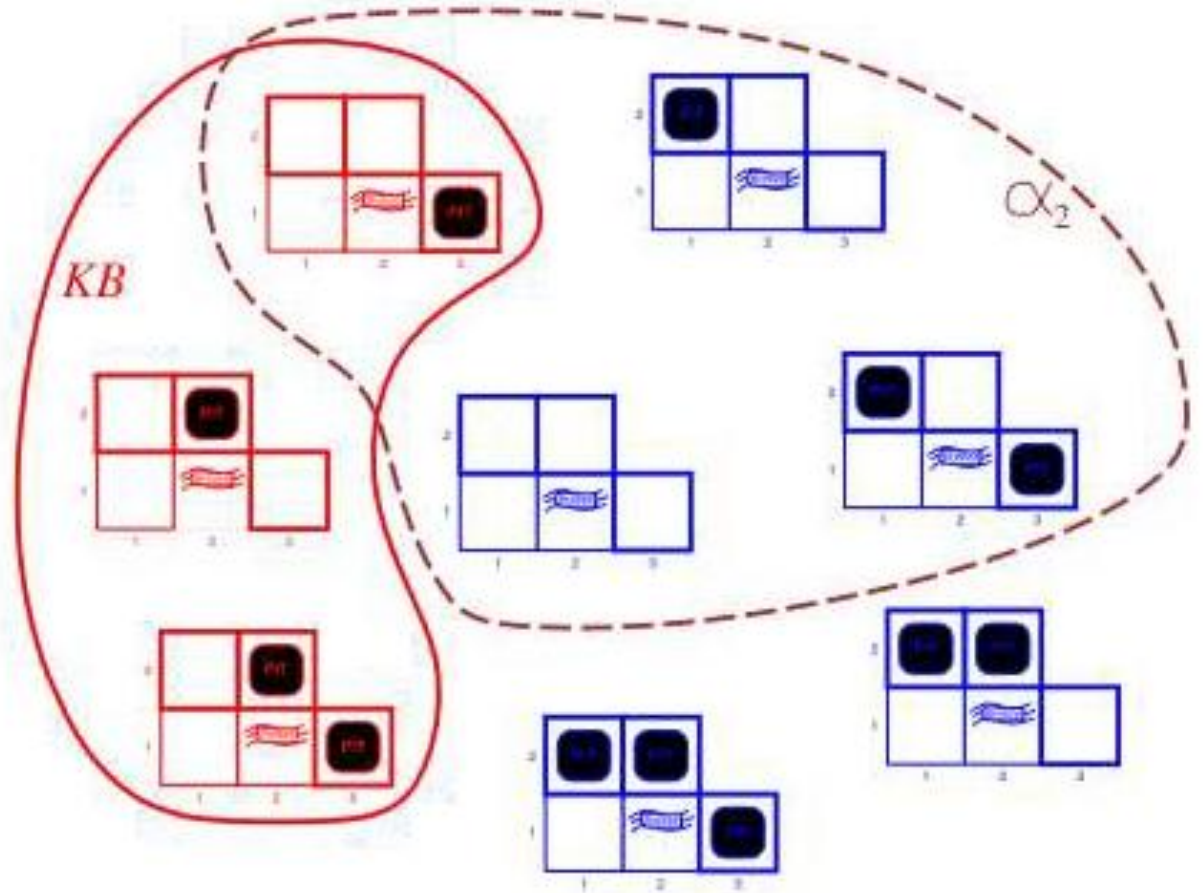
$KB \models \alpha_1$, proved by model checking



WUMPUS World Model

KB = world rules + observation

α_1 = "[2,2] is safe"
 $KB \vee \alpha_2$



Logical Inference

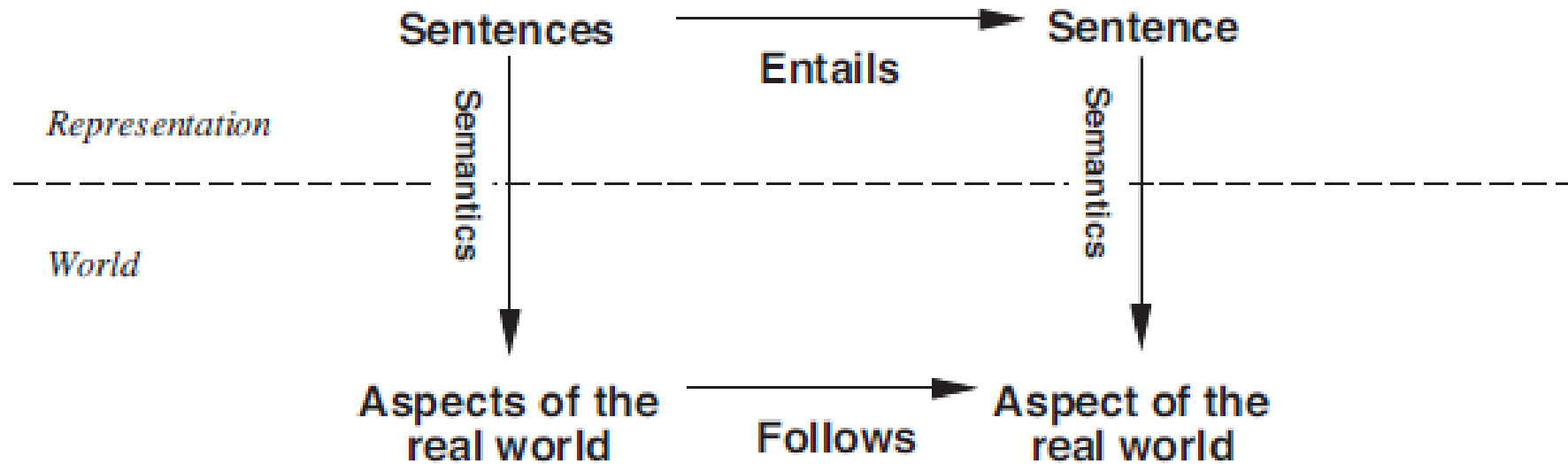
- The notion of entailment can be used for logic inference.
 - Model checking (see wumpus example): enumerate all possible models and check whether α is true.
- If an algorithm only derives entailed sentences it is called *sound* or *truth preserving*.
 - Otherwise it just makes things up.

i is sound if whenever $KB \vdash_i \alpha$ it is also true that $KB \models \alpha$

- **Completeness**: the algorithm can derive any sentence that is entailed.

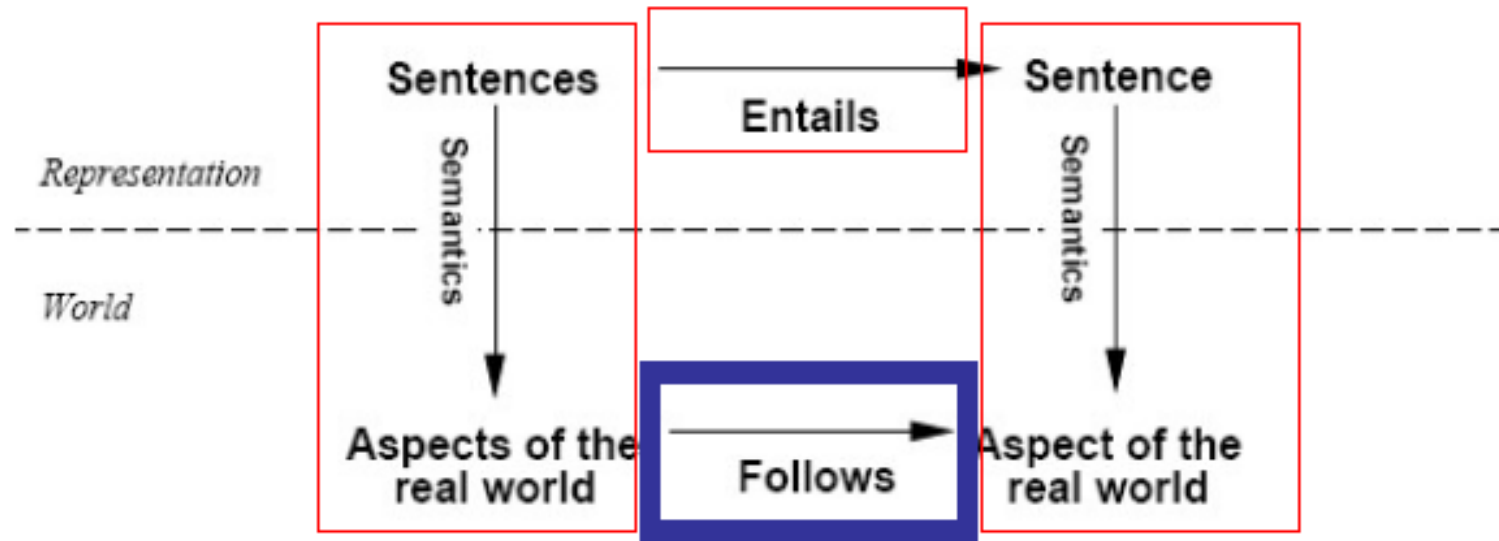
i is complete if whenever $KB \models \alpha$ it is also true that $KB \vdash_i \alpha$

Schematic perspective



- Sentences are physical configurations of the agent
- Reasoning is a process of constructing new physical configurations from old ones.
- Logical reasoning should ensure that the new configurations represent aspects of the world that actually follow from the aspects that the old configurations represent.

Schematic persepective



- If KB is true in the real world, then any sentence derived from KB by a sound inference procedure is also true in the real world.

Thank you

END OF WEEK 2



stablediffusionweb.com